

Cataract Canyon Trip Site

Workflow Guide: GitHub Pages · Google Photos · Claude Cowork

This guide covers how to integrate the three main tools for building and maintaining your river trip website: GitHub Pages for hosting, Google Photos for shared photo management, and Claude Cowork for building the HTML site.

1. Overall Architecture

The site uses a clean separation of concerns — each tool does what it's best at:

Component	Tool	Who manages it
Website hosting	GitHub Pages	You (via Claude Cowork)
Photo storage & uploads	Google Photos shared album	All group members
Site HTML/CSS/JS	Claude Cowork	You
Version control	Git / GitHub	Claude Cowork handles commands

2. GitHub Pages Setup (One-Time)

Do this once in the GitHub web interface — no command line needed.

2a. Create the repository

- Go to github.com and click **New repository**
- Name it something like **cataract-canyon-2026**
- Set visibility to **Public** (required for free GitHub Pages)
- Check **"Add a README file"** so the repo isn't empty
- Click **Create repository**

2b. Enable GitHub Pages

- In your new repo, click **Settings** (top tab bar)
- Scroll down to **Pages** in the left sidebar
- Under "Source", select **Deploy from a branch**
- Set branch to **main**, folder to **/ (root)**
- Click **Save**

Your site will be live within a minute or two at:

https://yourusername.github.io/cataract-canyon-2026

Note: Replace 'yourusername' with your actual GitHub username. The URL is live immediately but will show a 404 until you push an index.html.

3. Google Photos Setup — One Album Per Family

Google Photos does not support nested albums or subfolders — it is a flat structure. The solution is one shared album per family (six total). Each family uploads to their own album; your website gallery page presents all six with a card/button for each. This gives each family ownership of their photos and makes browsing more intentional.

3a. Create one album per family (repeat six times)

- Open **Google Photos** on your phone or at photos.google.com
- Tap **Library** → **Albums** → **Create album**
- Name it with the family name (e.g., "Cataract 2026 — Henderson Family")
- Tap the **share icon** and enable **Collaboration**
- Set to **Anyone with the link can add photos**
- Copy the share link — send it **only to that family**

Note: Tip: Do all six albums in one sitting. Keep a simple note with each family name and its corresponding Google Photos link — you'll need these when building gallery.html.

3b. Track your six album links

Family	Album Name	Google Photos Share Link
Family 1	Cataract 2026 — Family 1	https://photos.google.com/...
Family 2	Cataract 2026 — Family 2	https://photos.google.com/...
Family 3	Cataract 2026 — Family 3	https://photos.google.com/...
Family 4	Cataract 2026 — Family 4	https://photos.google.com/...
Family 5	Cataract 2026 — Family 5	https://photos.google.com/...
Family 6	Cataract 2026 — Family 6	https://photos.google.com/...

Note: Fill in the real URLs as you create each album. Pass this table to Claude Cowork when building gallery.html.

3c. How the gallery page works

Your gallery.html page presents six cards — one per family — each with the family name and a button linking to their Google Photos album. Claude Cowork can style these as a clean grid. Example brief:

"Build gallery.html with a 2-column card grid. Each card shows the family name, a short caption, and a 'View Photos' button linking to their Google Photos album. Use these six links: [paste your table of family names and URLs]. Match the site CSS. Push

when done."

Note: Google Photos does not support true iframe embeds for shared albums. Styled link buttons are the correct approach — they open the album in a new tab.

4. Local File Structure

Claude Cowork will create and manage files in a local folder on your Mac, then push changes to GitHub. Here's the recommended structure:

```
cataract-canyon-2026/ ■■■ index.html ← Landing page ■■■ river.html ← Natural
history of Cataract / Colorado R. ■■■ route.html ← Map and route details ■■■
triplog.html ← Day-by-day journal ■■■ gallery.html ← 6-card grid, one per family →
Google Photos ■■■ crew.html ← Group members page ■■■ css/ ■■■ style.css ← Shared
stylesheet ■■■ js/ ■■■ main.js ← Any interactivity ■■■ images/ ■■■ hero.jpg ←
Landing page hero image ■■■ highlights/ ← A few curated images (not full gallery)
```

Note: Keep the images/ folder lean — just hero and highlight images. Full photo library stays in Google Photos to avoid bloating the repo.

5. Working with Claude Cowork

Claude Cowork builds the site page by page. You describe what you want and it writes the HTML/CSS, manages git commits, and pushes to GitHub. You never need to type git commands yourself.

5a. First session — project setup

Give Claude Cowork a brief like this to start:

```
"Clone my GitHub repo cataract-canyon-2026 to a local folder. Create index.html with a
hero section, navigation bar linking to river.html, route.html, triplog.html,
gallery.html, and crew.html, and a css/style.css with a river/canyon color scheme.
Commit and push to GitHub when done."
```

5b. Subsequent sessions — building pages

Example brief for the natural history page:

```
"In river.html, create a page covering: the geology of Cataract Canyon
(Permian/Pennsylvanian strata), the Colorado River watershed and pre-dam vs. current
hydrology, Lake Powell and the ongoing drawdown, desert bighorn and peregrine falcon
ecology, the Powell expeditions of 1869 and 1871, and the Big Drop rapids lore. Use
the shared CSS. Push when done."
```

5c. Updating the site after the trip

To add new content or fix something:

```
"Open triplog.html. Add a Day 3 entry: [paste your notes here]. Commit with message
'Add Day 3 trip log' and push."
```

6. The Deploy Workflow (How Updates Go Live)

Every time Claude Cowork pushes a commit to the main branch, GitHub Pages automatically rebuilds and serves the updated site. Changes are typically live within 1–2 minutes.

Step	Action	Who/What
1	You describe a change to Claude Cowork	You
2	Claude Cowork edits the HTML/CSS files locally	Claude Cowork
3	Claude Cowork runs git add, commit, push	Claude Cowork
4	GitHub receives the push, triggers Pages build	GitHub (automatic)
5	Site is live at your .github.io URL	GitHub Pages

7. Maps Integration

For the route page, a Google Maps embed works well and is free. You can create a custom map with waypoints (Moab put-in, confluence, Big Drops, Hite takeout) and embed it directly.

Create a custom Google Map

- Go to **mymaps.google.com** and create a new map
- Add a layer for the river route (draw a line)
- Add point markers: put-in, campsites, Big Drop 1/2/3, takeout
- Click **Share** → **Anyone with link**, then click the 3-dot menu → **"Embed on my site"**
- Copy the iframe code and give it to Claude Cowork to drop into route.html

8. Quick Reference Checklist

Task	Where
GitHub repo created (cataract-canyon-2026)	github.com
GitHub Pages enabled (Settings → Pages → main branch)	github.com
Google Photos album created — Family 1 + link sent	photos.google.com
Google Photos album created — Family 2 + link sent	photos.google.com
Google Photos album created — Family 3 + link sent	photos.google.com
Google Photos album created — Family 4 + link sent	photos.google.com
Google Photos album created — Family 5 + link sent	photos.google.com
Google Photos album created — Family 6 + link sent	photos.google.com
All 6 album URLs collected in a note for Claude Cowork	Your notes

Local folder cloned by Claude Cowork	Claude Cowork
index.html + css/style.css created and pushed	Claude Cowork
river.html — natural history content	Claude Cowork
route.html — Google MyMaps embed	Claude Cowork + mymaps.google.com
triplog.html — day-by-day journal	Claude Cowork
gallery.html — 6-card grid linking to family albums	Claude Cowork
crew.html — group member page	Claude Cowork
Share site URL with the group	yourusername.github.io/cataract-canyon-2026